

Dynamic Programming

Dr. Bugra Caskurlu¹

¹ School of Computing
New Uzbekistan University

October, 2025

Motivating the Problem

Some Biology Lessons

- Double-stranded DNA is "zipped" together by complementary base-pairing.

Motivating the Problem

Some Biology Lessons

- Double-stranded DNA is "zipped" together by complementary base-pairing.
- Each strand of DNA can be viewed as a string of bases, where each base is drawn from the set $\{A, C, G, T\}$. The bases A and T pair with each other, and the bases C and G pair with each other; it is these $A - T$ and $C - G$ pairings that hold the two strands together.

Motivating the Problem

Some Biology Lessons

- Double-stranded DNA is "zipped" together by complementary base-pairing.
- Each strand of DNA can be viewed as a string of bases, where each base is drawn from the set $\{A, C, G, T\}$. The bases A and T pair with each other, and the bases C and G pair with each other; it is these $A - T$ and $C - G$ pairings that hold the two strands together.
- Single-stranded RNA molecules are key components in many of the processes that go on inside a cell, and they follow more or less the same structural principles.

Motivating the Problem

Some Biology Lessons

- Double-stranded DNA is "zipped" together by complementary base-pairing.
- Each strand of DNA can be viewed as a string of bases, where each base is drawn from the set $\{A, C, G, T\}$. The bases A and T pair with each other, and the bases C and G pair with each other; it is these $A - T$ and $C - G$ pairings that hold the two strands together.
- Single-stranded RNA molecules are key components in many of the processes that go on inside a cell, and they follow more or less the same structural principles.
- However, unlike double-stranded DNA, there's no "second strand" for the RNA to stick to; so it tends to loop back and form base pairs with itself, resulting in interesting shapes.

Motivating the Problem

Some Biology Lessons

- Double-stranded DNA is "zipped" together by complementary base-pairing.
- Each strand of DNA can be viewed as a string of bases, where each base is drawn from the set $\{A, C, G, T\}$. The bases A and T pair with each other, and the bases C and G pair with each other; it is these $A - T$ and $C - G$ pairings that hold the two strands together.
- Single-stranded RNA molecules are key components in many of the processes that go on inside a cell, and they follow more or less the same structural principles.
- However, unlike double-stranded DNA, there's no "second strand" for the RNA to stick to; so it tends to loop back and form base pairs with itself, resulting in interesting shapes.
- The set of pairs (and resulting shape) formed by the RNA molecule through this process is called the secondary structure, and understanding the secondary structure is essential for understanding the behavior of the molecule.

Motivating the Problem

Some Biology Lessons

- Double-stranded DNA is "zipped" together by complementary base-pairing.
- Each strand of DNA can be viewed as a string of bases, where each base is drawn from the set $\{A, C, G, T\}$. The bases A and T pair with each other, and the bases C and G pair with each other; it is these $A - T$ and $C - G$ pairings that hold the two strands together.
- Single-stranded RNA molecules are key components in many of the processes that go on inside a cell, and they follow more or less the same structural principles.
- However, unlike double-stranded DNA, there's no "second strand" for the RNA to stick to; so it tends to loop back and form base pairs with itself, resulting in interesting shapes.
- The set of pairs (and resulting shape) formed by the RNA molecule through this process is called the secondary structure, and understanding the secondary structure is essential for understanding the behavior of the molecule.
- A single-stranded RNA molecule can be viewed as a sequence of n symbols (bases) drawn from the alphabet $\{A, C, G, U\}$.

Motivating the Problem

Some Biology Lessons

- Double-stranded DNA is "zipped" together by complementary base-pairing.
- Each strand of DNA can be viewed as a string of bases, where each base is drawn from the set $\{A, C, G, T\}$. The bases A and T pair with each other, and the bases C and G pair with each other; it is these $A - T$ and $C - G$ pairings that hold the two strands together.
- Single-stranded RNA molecules are key components in many of the processes that go on inside a cell, and they follow more or less the same structural principles.
- However, unlike double-stranded DNA, there's no "second strand" for the RNA to stick to; so it tends to loop back and form base pairs with itself, resulting in interesting shapes.
- The set of pairs (and resulting shape) formed by the RNA molecule through this process is called the secondary structure, and understanding the secondary structure is essential for understanding the behavior of the molecule.
- A single-stranded RNA molecule can be viewed as a sequence of n symbols (bases) drawn from the alphabet $\{A, C, G, U\}$.
- Let $B = b_1 b_2 \dots b_n$ be a single-stranded RNA molecule, where each $b_i \in \{A, C, G, U\}$.

Motivating the Problem

Some Biology Lessons

- Double-stranded DNA is "zipped" together by complementary base-pairing.
- Each strand of DNA can be viewed as a string of bases, where each base is drawn from the set $\{A, C, G, T\}$. The bases A and T pair with each other, and the bases C and G pair with each other; it is these $A - T$ and $C - G$ pairings that hold the two strands together.
- Single-stranded RNA molecules are key components in many of the processes that go on inside a cell, and they follow more or less the same structural principles.
- However, unlike double-stranded DNA, there's no "second strand" for the RNA to stick to; so it tends to loop back and form base pairs with itself, resulting in interesting shapes.
- The set of pairs (and resulting shape) formed by the RNA molecule through this process is called the secondary structure, and understanding the secondary structure is essential for understanding the behavior of the molecule.
- A single-stranded RNA molecule can be viewed as a sequence of n symbols (bases) drawn from the alphabet $\{A, C, G, U\}$.
- Let $B = b_1 b_2 \dots b_n$ be a single-stranded RNA molecule, where each $b_i \in \{A, C, G, U\}$.
- To a *first approximation*, one can model its secondary structure as follows:

Motivating the Problem

Some Biology Lessons

- Double-stranded DNA is "zipped" together by complementary base-pairing.
- Each strand of DNA can be viewed as a string of bases, where each base is drawn from the set $\{A, C, G, T\}$. The bases A and T pair with each other, and the bases C and G pair with each other; it is these $A - T$ and $C - G$ pairings that hold the two strands together.
- Single-stranded RNA molecules are key components in many of the processes that go on inside a cell, and they follow more or less the same structural principles.
- However, unlike double-stranded DNA, there's no "second strand" for the RNA to stick to; so it tends to loop back and form base pairs with itself, resulting in interesting shapes.
- The set of pairs (and resulting shape) formed by the RNA molecule through this process is called the secondary structure, and understanding the secondary structure is essential for understanding the behavior of the molecule.
- A single-stranded RNA molecule can be viewed as a sequence of n symbols (bases) drawn from the alphabet $\{A, C, G, U\}$.
- Let $B = b_1 b_2 \dots b_n$ be a single-stranded RNA molecule, where each $b_i \in \{A, C, G, U\}$.
- To a *first approximation*, one can model its secondary structure as follows:
 - ① A pairs with U , and C pairs with G ; and each base can pair with at most one other base.

Motivating the Problem

Some Biology Lessons

- Double-stranded DNA is "zipped" together by complementary base-pairing.
- Each strand of DNA can be viewed as a string of bases, where each base is drawn from the set $\{A, C, G, T\}$. The bases A and T pair with each other, and the bases C and G pair with each other; it is these $A - T$ and $C - G$ pairings that hold the two strands together.
- Single-stranded RNA molecules are key components in many of the processes that go on inside a cell, and they follow more or less the same structural principles.
- However, unlike double-stranded DNA, there's no "second strand" for the RNA to stick to; so it tends to loop back and form base pairs with itself, resulting in interesting shapes.
- The set of pairs (and resulting shape) formed by the RNA molecule through this process is called the secondary structure, and understanding the secondary structure is essential for understanding the behavior of the molecule.
- A single-stranded RNA molecule can be viewed as a sequence of n symbols (bases) drawn from the alphabet $\{A, C, G, U\}$.
- Let $B = b_1 b_2 \dots b_n$ be a single-stranded RNA molecule, where each $b_i \in \{A, C, G, U\}$.
- To a *first approximation*, one can model its secondary structure as follows:
 - 1 A pairs with U , and C pairs with G ; and each base can pair with at most one other base.
 - 2 It also turns out that secondary structures are "knot-free".

An RNA Secondary Structure Example

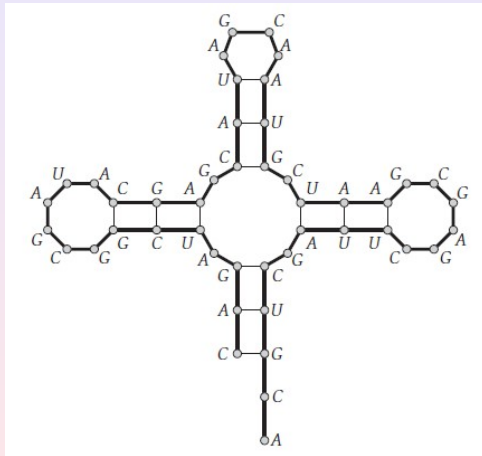


Figure: An RNA secondary structure. Thick lines connect adjacent elements of the sequence; thin lines indicate pairs of elements that are matched.

RNA Secondary Structure Problem

Formal Problem Definition

- A *secondary structure* on B is a set of pairs $S = \{(i, j)\}$, where $i, j \in \{1, 2, \dots, n\}$, that satisfies the following conditions:
 - ❶ No sharp turns: The ends of each pair in S are separated by at least four intervening bases; that is, if $(i, j) \in S$, then $i < j - 4$.

RNA Secondary Structure Problem

Formal Problem Definition

- A *secondary structure* on B is a set of pairs $S = \{(i, j)\}$, where $i, j \in \{1, 2, \dots, n\}$, that satisfies the following conditions:
 - 1 No sharp turns: The ends of each pair in S are separated by at least four intervening bases; that is, if $(i, j) \in S$, then $i < j - 4$.
 - 2 The elements of any pair in S consist of either $\{A, U\}$ or $\{C, G\}$ (in either order).

RNA Secondary Structure Problem

Formal Problem Definition

- A *secondary structure* on B is a set of pairs $S = \{(i, j)\}$, where $i, j \in \{1, 2, \dots, n\}$, that satisfies the following conditions:
 - 1 No sharp turns: The ends of each pair in S are separated by at least four intervening bases; that is, if $(i, j) \in S$, then $i < j - 4$.
 - 2 The elements of any pair in S consist of either $\{A, U\}$ or $\{C, G\}$ (in either order).
 - 3 S is a matching: no base appears in more than one pair.

RNA Secondary Structure Problem

Formal Problem Definition

- A *secondary structure* on B is a set of pairs $S = \{(i, j)\}$, where $i, j \in \{1, 2, \dots, n\}$, that satisfies the following conditions:
 - 1 No sharp turns: The ends of each pair in S are separated by at least four intervening bases; that is, if $(i, j) \in S$, then $i < j - 4$.
 - 2 The elements of any pair in S consist of either $\{A, U\}$ or $\{C, G\}$ (in either order).
 - 3 S is a matching: no base appears in more than one pair.
 - 4 The noncrossing condition: If (i, j) and (k, l) are two pairs in S , then we cannot have $i < k < j < l$.

RNA Secondary Structure Problem

Formal Problem Definition

- A *secondary structure* on B is a set of pairs $S = \{(i, j)\}$, where $i, j \in \{1, 2, \dots, n\}$, that satisfies the following conditions:
 - ❶ No sharp turns: The ends of each pair in S are separated by at least four intervening bases; that is, if $(i, j) \in S$, then $i < j - 4$.
 - ❷ The elements of any pair in S consist of either $\{A, U\}$ or $\{C, G\}$ (in either order).
 - ❸ S is a matching: no base appears in more than one pair.
 - ❹ The noncrossing condition: If (i, j) and (k, l) are two pairs in S , then we cannot have $i < k < j < l$.
- We want an efficient algorithm that takes a single-stranded RNA molecule $B = b_1 b_2 \dots b_n$ and determines a secondary structure S with the maximum possible number of base pairs.

RNA Secondary Structure Problem

Formal Problem Definition

- A *secondary structure* on B is a set of pairs $S = \{(i, j)\}$, where $i, j \in \{1, 2, \dots, n\}$, that satisfies the following conditions:
 - 1 No sharp turns: The ends of each pair in S are separated by at least four intervening bases; that is, if $(i, j) \in S$, then $i < j - 4$.
 - 2 The elements of any pair in S consist of either $\{A, U\}$ or $\{C, G\}$ (in either order).
 - 3 S is a matching: no base appears in more than one pair.
 - 4 The noncrossing condition: If (i, j) and (k, l) are two pairs in S , then we cannot have $i < k < j < l$.
- We want an efficient algorithm that takes a single-stranded RNA molecule $B = b_1 b_2 \dots b_n$ and determines a secondary structure S with the maximum possible number of base pairs.

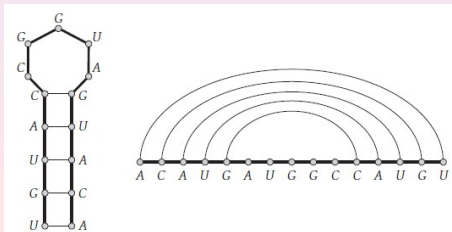


Figure: Two views of an RNA secondary structure. In the second view, the string has been "stretched" lengthwise, and edges connecting matched pairs appear as noncrossing "bubbles" over the string.

A False Attempt

Let's Try What Has Worked In Other Examples

- The natural first attempt to apply dynamic programming would presumably be based on the following sub-problems:

A False Attempt

Let's Try What Has Worked In Other Examples

- The natural first attempt to apply dynamic programming would presumably be based on the following sub-problems:
- We say that $OPT(j)$ is the maximum number of base pairs in a secondary structure on $b_1 b_2 \dots b_j$.

A False Attempt

Let's Try What Has Worked In Other Examples

- The natural first attempt to apply dynamic programming would presumably be based on the following sub-problems:
- We say that $OPT(j)$ is the maximum number of base pairs in a secondary structure on $b_1 b_2 \dots b_j$.
- By the no-sharp-turns condition, we know that $OPT(j) = 0$ for $j \leq 5$; and we know that $OPT(n)$ is the solution we're looking for.

A False Attempt

Let's Try What Has Worked In Other Examples

- The natural first attempt to apply dynamic programming would presumably be based on the following sub-problems:
- We say that $OPT(j)$ is the maximum number of base pairs in a secondary structure on $b_1 b_2 \dots b_j$.
- By the no-sharp-turns condition, we know that $OPT(j) = 0$ for $j \leq 5$; and we know that $OPT(n)$ is the solution we're looking for.
- The trouble comes when we try writing down a recurrence that expresses $OPT(j)$ in terms of the solutions to smaller sub-problems.

A False Attempt

Let's Try What Has Worked In Other Examples

- The natural first attempt to apply dynamic programming would presumably be based on the following sub-problems:
- We say that $OPT(j)$ is the maximum number of base pairs in a secondary structure on $b_1 b_2 \dots b_j$.
- By the no-sharp-turns condition, we know that $OPT(j) = 0$ for $j \leq 5$; and we know that $OPT(n)$ is the solution we're looking for.
- The trouble comes when we try writing down a recurrence that expresses $OPT(j)$ in terms of the solutions to smaller sub-problems.
- We can get partway there: in the optimal secondary structure on $b_1 b_2 \dots b_j$, it's the case that either j is not involved in a pair; or j pairs with t for some $t < j - 4$.

A False Attempt

Let's Try What Has Worked In Other Examples

- The natural first attempt to apply dynamic programming would presumably be based on the following sub-problems:
- We say that $OPT(j)$ is the maximum number of base pairs in a secondary structure on $b_1 b_2 \dots b_j$.
- By the no-sharp-turns condition, we know that $OPT(j) = 0$ for $j \leq 5$; and we know that $OPT(n)$ is the solution we're looking for.
- The trouble comes when we try writing down a recurrence that expresses $OPT(j)$ in terms of the solutions to smaller sub-problems.
- We can get partway there: in the optimal secondary structure on $b_1 b_2 \dots b_j$, it's the case that either j is not involved in a pair; or j pairs with t for some $t < j - 4$.
- In the first case, we just need to consult our solution for $OPT(j - 1)$.

A False Attempt

Let's Try What Has Worked In Other Examples

- The natural first attempt to apply dynamic programming would presumably be based on the following sub-problems:
- We say that $OPT(j)$ is the maximum number of base pairs in a secondary structure on $b_1 b_2 \dots b_j$.
- By the no-sharp-turns condition, we know that $OPT(j) = 0$ for $j \leq 5$; and we know that $OPT(n)$ is the solution we're looking for.
- The trouble comes when we try writing down a recurrence that expresses $OPT(j)$ in terms of the solutions to smaller sub-problems.
- We can get partway there: in the optimal secondary structure on $b_1 b_2 \dots b_j$, it's the case that either j is not involved in a pair; or j pairs with t for some $t < j - 4$.
- In the first case, we just need to consult our solution for $OPT(j - 1)$.
- On the second case; because of the non-crossing condition, we now know that no pair can have one end between 1 and $t - 1$ and the other end between $t + 1$ and $j - 1$.

A False Attempt

Let's Try What Has Worked In Other Examples

- The natural first attempt to apply dynamic programming would presumably be based on the following sub-problems:
- We say that $OPT(j)$ is the maximum number of base pairs in a secondary structure on $b_1 b_2 \dots b_j$.
- By the no-sharp-turns condition, we know that $OPT(j) = 0$ for $j \leq 5$; and we know that $OPT(n)$ is the solution we're looking for.
- The trouble comes when we try writing down a recurrence that expresses $OPT(j)$ in terms of the solutions to smaller sub-problems.
- We can get partway there: in the optimal secondary structure on $b_1 b_2 \dots b_j$, it's the case that either j is not involved in a pair; or j pairs with t for some $t < j - 4$.
- In the first case, we just need to consult our solution for $OPT(j - 1)$.
- On the second case; because of the non-crossing condition, we now know that no pair can have one end between 1 and $t - 1$ and the other end between $t + 1$ and $j - 1$.
- We've therefore effectively isolated two new sub-problems: one on the bases $b_1 b_2 \dots b_{t-1}$, and the other on the bases $b_{t+1} \dots b_{j-1}$. The first is solved by $OPT(t - 1)$, but the second is not on our list of sub-problems, because it does not begin with b_1 .

Identifying Sub-problems

Including the pair (t, j) results in two independent subproblems.

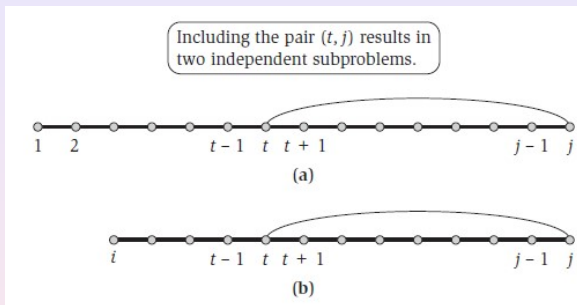


(a)



(b)

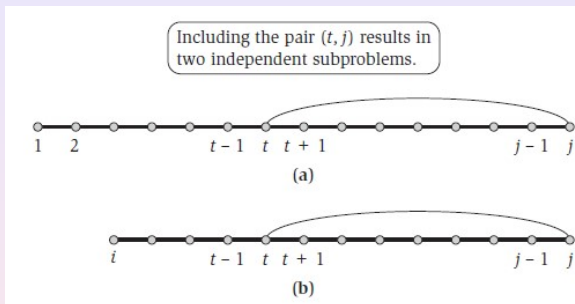
Identifying Sub-problems



Intervals As Sub-problems

- We need to be able to work with sub-problems that do not begin with b_1 ; in other words, we need to consider sub-problems on $b_i b_{i+1} \dots b_j$ for all choices of $i \leq j$.

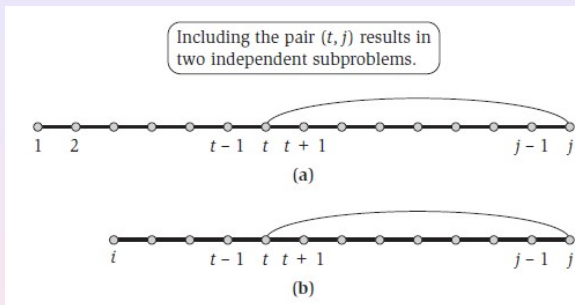
Identifying Sub-problems



Intervals As Sub-problems

- We need to be able to work with sub-problems that do not begin with b_1 ; in other words, we need to consider sub-problems on $b_i b_{i+1} \dots b_j$ for all choices of $i \leq j$.
- Let $OPT(i, j)$ denote the maximum number of base pairs in a secondary structure on $b_i b_{i+1} \dots b_j$.

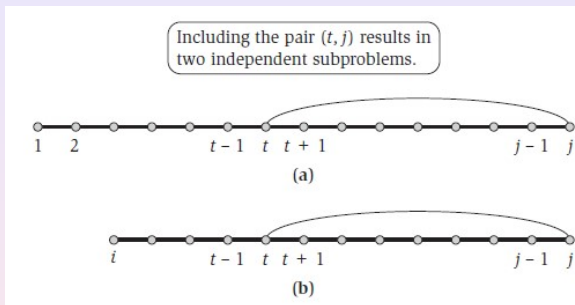
Identifying Sub-problems



Intervals As Sub-problems

- We need to be able to work with sub-problems that do not begin with b_1 ; in other words, we need to consider sub-problems on $b_i b_{i+1} \dots b_j$ for all choices of $i \leq j$.
- Let $OPT(i, j)$ denote the maximum number of base pairs in a secondary structure on $b_i b_{i+1} \dots b_j$.
- The no-sharp-turns condition lets us initialize $OPT(i, j) = 0$ whenever $i \geq j - 4$. For notational convenience, we allow ourselves to refer to $OPT(i, j)$ even when $i > j$; in this case, its value is 0.

Identifying Sub-problems



Intervals As Sub-problems

- We need to be able to work with sub-problems that do not begin with b_1 ; in other words, we need to consider sub-problems on $b_i b_{i+1} \dots b_j$ for all choices of $i \leq j$.
- Let $OPT(i, j)$ denote the maximum number of base pairs in a secondary structure on $b_i b_{i+1} \dots b_j$.
- The no-sharp-turns condition lets us initialize $OPT(i, j) = 0$ whenever $i \geq j - 4$. For notational convenience, we allow ourselves to refer to $OPT(i, j)$ even when $i > j$; in this case, its value is 0.

Obtaining The Recurrence

Explanation

- Now, in the optimal secondary structure on $b_i b_{i+1} \dots b_j$, we have the same alternatives as before:

Obtaining The Recurrence

Explanation

- Now, in the optimal secondary structure on $b_i b_{i+1} \dots b_j$, we have the same alternatives as before:
 - 1 j is not involved in a pair; or

Obtaining The Recurrence

Explanation

- Now, in the optimal secondary structure on $b_i b_{i+1} \dots b_j$, we have the same alternatives as before:
 - ① j is not involved in a pair; or
 - ② j pairs with t for some $t < j - 4$.

Obtaining The Recurrence

Explanation

- Now, in the optimal secondary structure on $b_i b_{i+1} \dots b_j$, we have the same alternatives as before:
 - ① j is not involved in a pair; or
 - ② j pairs with t for some $t < j - 4$.
- In the first case, we have $OPT(i, j) = OPT(i, j - 1)$.

Obtaining The Recurrence

Explanation

- Now, in the optimal secondary structure on $b_i b_{i+1} \dots b_j$, we have the same alternatives as before:
 - ① j is not involved in a pair; or
 - ② j pairs with t for some $t < j - 4$.
- In the first case, we have $OPT(i, j) = OPT(i, j - 1)$.
- In the second case, we recur on the two sub-problems $OPT(i, t - 1)$ and $OPT(t + 1, j - 1)$; the non-crossing condition has isolated these two sub-problems from each other.

Obtaining The Recurrence

Explanation

- Now, in the optimal secondary structure on $b_i b_{i+1} \dots b_j$, we have the same alternatives as before:
 - 1 j is not involved in a pair; or
 - 2 j pairs with t for some $t < j - 4$.
- In the first case, we have $OPT(i, j) = OPT(i, j - 1)$.
- In the second case, we recur on the two sub-problems $OPT(i, t - 1)$ and $OPT(t + 1, j - 1)$; the non-crossing condition has isolated these two sub-problems from each other.
- We have therefore justified the recurrence:

$$OPT(i, j) = \max(OPT(i, j - 1), \max_t(1 + OPT(i, t - 1) + OPT(t + 1, j - 1))),$$

where the max is taken over t such that b_t and b_j are an allowable base pair (under conditions (1) and (2) from the definition of a secondary structure).

Obtaining The Recurrence

Explanation

- Now, in the optimal secondary structure on $b_i b_{i+1} \dots b_j$, we have the same alternatives as before:
 - ① j is not involved in a pair; or
 - ② j pairs with t for some $t < j - 4$.
- In the first case, we have $OPT(i, j) = OPT(i, j - 1)$.
- In the second case, we recur on the two sub-problems $OPT(i, t - 1)$ and $OPT(t + 1, j - 1)$; the non-crossing condition has isolated these two sub-problems from each other.
- We have therefore justified the recurrence:

$$OPT(i, j) = \max(OPT(i, j - 1), \max_t(1 + OPT(i, t - 1) + OPT(t + 1, j - 1))),$$

where the max is taken over t such that b_t and b_j are an allowable base pair (under conditions (1) and (2) from the definition of a secondary structure).

- Now we just have to make sure we understand the proper order in which to build up the solutions to the sub-problems.

Obtaining The Recurrence

Explanation

- Now, in the optimal secondary structure on $b_i b_{i+1} \dots b_j$, we have the same alternatives as before:
 - 1 j is not involved in a pair; or
 - 2 j pairs with t for some $t < j - 4$.
- In the first case, we have $OPT(i, j) = OPT(i, j - 1)$.
- In the second case, we recur on the two sub-problems $OPT(i, t - 1)$ and $OPT(t + 1, j - 1)$; the non-crossing condition has isolated these two sub-problems from each other.
- We have therefore justified the recurrence:

$$OPT(i, j) = \max(OPT(i, j - 1), \max_t (1 + OPT(i, t - 1) + OPT(t + 1, j - 1))),$$

where the max is taken over t such that b_t and b_j are an allowable base pair (under conditions (1) and (2) from the definition of a secondary structure).

- Now we just have to make sure we understand the proper order in which to build up the solutions to the sub-problems.
- The form of recurrence reveals that we're always invoking the solution to sub-problems on shorter intervals: those for which $k = j - i$ is smaller.

Obtaining The Recurrence

Explanation

- Now, in the optimal secondary structure on $b_i b_{i+1} \dots b_j$, we have the same alternatives as before:
 - 1 j is not involved in a pair; or
 - 2 j pairs with t for some $t < j - 4$.
- In the first case, we have $OPT(i, j) = OPT(i, j - 1)$.
- In the second case, we recur on the two sub-problems $OPT(i, t - 1)$ and $OPT(t + 1, j - 1)$; the non-crossing condition has isolated these two sub-problems from each other.
- We have therefore justified the recurrence:

$$OPT(i, j) = \max(OPT(i, j - 1), \max_t (1 + OPT(i, t - 1) + OPT(t + 1, j - 1))),$$

where the max is taken over t such that b_t and b_j are an allowable base pair (under conditions (1) and (2) from the definition of a secondary structure).

- Now we just have to make sure we understand the proper order in which to build up the solutions to the sub-problems.
- The form of recurrence reveals that we're always invoking the solution to sub-problems on shorter intervals: those for which $k = j - i$ is smaller.
- Thus things will work without any trouble if we build up the solutions in order of increasing interval length.

Pseudocode for Dynamic Programming Solution

Recurrence (6.13)

$$OPT(i, j) = \max(OPT(i, j-1), \max(1 + OPT(i, t-1) + OPT(t+1, j-1))),$$

where the max is taken over t such that b_t and b_j are an allowable base pair (under conditions (1) and (2) from the definition of a secondary structure).

Pseudocode for Dynamic Programming Solution

Recurrence (6.13)

$$OPT(i, j) = \max(OPT(i, j-1), \max(1 + OPT(i, t-1) + OPT(t+1, j-1))),$$

where the max is taken over t such that b_t and b_j are an allowable base pair (under conditions (1) and (2) from the definition of a secondary structure).

```
Initialize  $OPT(i, j) = 0$  whenever  $i \geq j - 4$ 
For  $k = 5, 6, \dots, n - 1$ 
  For  $i = 1, 2, \dots, n - k$ 
    Set  $j = i + k$ 
    Compute  $OPT(i, j)$  using the recurrence in (6.13)
  Endfor
Endfor
Return  $OPT(1, n)$ 
```

Matrix-Chain Multiplication

Problem Definition

- Our next example of dynamic programming is an algorithm that solves the problem of matrix-chain multiplication.

Matrix-Chain Multiplication

Problem Definition

- Our next example of dynamic programming is an algorithm that solves the problem of matrix-chain multiplication.
- Given a sequence (chain) $\langle A_1, A_2, \dots, A_n \rangle$ of n matrices to be multiplied, where the matrices aren't necessarily square, the goal is to compute the product $A_1 A_2 \dots A_n$ using the standard algorithm for multiplying rectangular matrices, while minimizing the number of scalar multiplications.

Matrix-Chain Multiplication

Problem Definition

- Our next example of dynamic programming is an algorithm that solves the problem of matrix-chain multiplication.
- Given a sequence (chain) $\langle A_1, A_2, \dots, A_n \rangle$ of n matrices to be multiplied, where the matrices aren't necessarily square, the goal is to compute the product $A_1 A_2 \dots A_n$ using the standard algorithm for multiplying rectangular matrices, while minimizing the number of scalar multiplications.

RECTANGULAR-MATRIX-MULTIPLY(A, B, C, p, q, r)

```
1  for  $i = 1$  to  $p$ 
2      for  $j = 1$  to  $r$ 
3          for  $k = 1$  to  $q$ 
4               $c_{ij} = c_{ij} + a_{ik} \cdot b_{kj}$ 
```

Figure: Computes $C = C + A \cdot B$ for three matrices $A = (a_{ij})$, $B = (b_{ij})$, and $C = (c_{ij})$, where A is $p \times q$, B is $q \times r$, and C is $p \times r$.

Explaining the Problem

Details

- We can evaluate $A_1 A_2 \dots A_n$ using the algorithm for multiplying pairs of rectangular matrices as a subroutine once we have parenthesized it to resolve all ambiguities in how the matrices are multiplied together.

Explaining the Problem

Details

- We can evaluate $A_1 A_2 \dots A_n$ using the algorithm for multiplying pairs of rectangular matrices as a subroutine once we have parenthesized it to resolve all ambiguities in how the matrices are multiplied together.
- Matrix multiplication is associative, and so all parenthesizations yield the same product.

Explaining the Problem

Details

- We can evaluate $A_1 A_2 \dots A_n$ using the algorithm for multiplying pairs of rectangular matrices as a subroutine once we have parenthesized it to resolve all ambiguities in how the matrices are multiplied together.
- Matrix multiplication is associative, and so all parenthesizations yield the same product.
- A product of matrices is **fully parenthesized** if it is either a single matrix or the product of two fully parenthesized matrix products, surrounded by parentheses.

Explaining the Problem

Details

- We can evaluate $A_1 A_2 \dots A_n$ using the algorithm for multiplying pairs of rectangular matrices as a subroutine once we have parenthesized it to resolve all ambiguities in how the matrices are multiplied together.
- Matrix multiplication is associative, and so all parenthesizations yield the same product.
- A product of matrices is **fully parenthesized** if it is either a single matrix or the product of two fully parenthesized matrix products, surrounded by parentheses.
- For example, if the chain of matrices is $\langle A_1, A_2, A_3, A_4 \rangle$, then you can fully parenthesize the product $A_1 A_2 A_3 A_4$ in five distinct ways:

Explaining the Problem

Details

- We can evaluate $A_1 A_2 \dots A_n$ using the algorithm for multiplying pairs of rectangular matrices as a subroutine once we have parenthesized it to resolve all ambiguities in how the matrices are multiplied together.
- Matrix multiplication is associative, and so all parenthesizations yield the same product.
- A product of matrices is **fully parenthesized** if it is either a single matrix or the product of two fully parenthesized matrix products, surrounded by parentheses.
- For example, if the chain of matrices is $\langle A_1, A_2, A_3, A_4 \rangle$, then you can fully parenthesize the product $A_1 A_2 A_3 A_4$ in five distinct ways:
 - 1 $(A_1 (A_2 (A_3 A_4)))$,
 - 2 $(A_1 ((A_2 A_3) A_4))$,
 - 3 $((A_1 A_2) (A_3 A_4))$,
 - 4 $((A_1 (A_2 A_3)) A_4)$,
 - 5 $(((A_1 A_2) A_3) A_4)$.

The Number of Scalar Multiplications Depends on the Parenthesization

Explanation Through an Example

- How you parenthesize a chain of matrices can have a dramatic impact on the cost of evaluating the product.

The Number of Scalar Multiplications Depends on the Parenthesization

Explanation Through an Example

- How you parenthesize a chain of matrices can have a dramatic impact on the cost of evaluating the product.
- Notice that multiplying a $p \times q$ matrix A with a $q \times r$ matrix B requires pqr scalar multiplications.

The Number of Scalar Multiplications Depends on the Parenthesization

Explanation Through an Example

- How you parenthesize a chain of matrices can have a dramatic impact on the cost of evaluating the product.
- Notice that multiplying a $p \times q$ matrix A with a $q \times r$ matrix B requires pqr scalar multiplications.
- To illustrate the different costs incurred by different parenthesizations of a matrix product, consider the problem of a chain $\langle A_1, A_2, A_3 \rangle$ of three matrices.

The Number of Scalar Multiplications Depends on the Parenthesization

Explanation Through an Example

- How you parenthesize a chain of matrices can have a dramatic impact on the cost of evaluating the product.
- Notice that multiplying a $p \times q$ matrix A with a $q \times r$ matrix B requires pqr scalar multiplications.
- To illustrate the different costs incurred by different parenthesizations of a matrix product, consider the problem of a chain $\langle A_1, A_2, A_3 \rangle$ of three matrices.
- Suppose that the dimensions of the matrices are 10×100 , 100×5 , and 5×50 , respectively.

The Number of Scalar Multiplications Depends on the Parenthesization

Explanation Through an Example

- How you parenthesize a chain of matrices can have a dramatic impact on the cost of evaluating the product.
- Notice that multiplying a $p \times q$ matrix A with a $q \times r$ matrix B requires pqr scalar multiplications.
- To illustrate the different costs incurred by different parenthesizations of a matrix product, consider the problem of a chain $\langle A_1, A_2, A_3 \rangle$ of three matrices.
- Suppose that the dimensions of the matrices are 10×100 , 100×5 , and 5×50 , respectively.
- Multiplying according to the parenthesization $((A_1 A_2) A_3)$ performs $10 \cdot 100 \cdot 5 = 5000$ scalar multiplications to compute the 10×5 matrix product $A_1 A_2$, plus another $10 \cdot 5 \cdot 50 = 2500$ scalar multiplications to multiply this matrix by A_3 , for a total of 7500 scalar multiplications.

The Number of Scalar Multiplications Depends on the Parenthesization

Explanation Through an Example

- How you parenthesize a chain of matrices can have a dramatic impact on the cost of evaluating the product.
- Notice that multiplying a $p \times q$ matrix A with a $q \times r$ matrix B requires pqr scalar multiplications.
- To illustrate the different costs incurred by different parenthesizations of a matrix product, consider the problem of a chain $\langle A_1, A_2, A_3 \rangle$ of three matrices.
- Suppose that the dimensions of the matrices are 10×100 , 100×5 , and 5×50 , respectively.
- Multiplying according to the parenthesization $((A_1 A_2) A_3)$ performs $10 \cdot 100 \cdot 5 = 5000$ scalar multiplications to compute the 10×5 matrix product $A_1 A_2$, plus another $10 \cdot 5 \cdot 50 = 2500$ scalar multiplications to multiply this matrix by A_3 , for a total of 7500 scalar multiplications.
- Multiplying according to the alternative parenthesization $(A_1 (A_2 A_3))$ performs $100 \cdot 5 \cdot 50 = 25,000$ scalar multiplications to compute the 100×50 matrix product $A_2 A_3$, plus another $10 \cdot 100 \cdot 50 = 50,000$ scalar multiplications to multiply A_1 by this matrix, for a total of 75,000 scalar multiplications.

The Number of Scalar Multiplications Depends on the Parenthesization

Explanation Through an Example

- How you parenthesize a chain of matrices can have a dramatic impact on the cost of evaluating the product.
- Notice that multiplying a $p \times q$ matrix A with a $q \times r$ matrix B requires pqr scalar multiplications.
- To illustrate the different costs incurred by different parenthesizations of a matrix product, consider the problem of a chain $\langle A_1, A_2, A_3 \rangle$ of three matrices.
- Suppose that the dimensions of the matrices are 10×100 , 100×5 , and 5×50 , respectively.
- Multiplying according to the parenthesization $((A_1 A_2) A_3)$ performs $10 \cdot 100 \cdot 5 = 5000$ scalar multiplications to compute the 10×5 matrix product $A_1 A_2$, plus another $10 \cdot 5 \cdot 50 = 2500$ scalar multiplications to multiply this matrix by A_3 , for a total of 7500 scalar multiplications.
- Multiplying according to the alternative parenthesization $(A_1 (A_2 A_3))$ performs $100 \cdot 5 \cdot 50 = 25,000$ scalar multiplications to compute the 100×50 matrix product $A_2 A_3$, plus another $10 \cdot 100 \cdot 50 = 50,000$ scalar multiplications to multiply A_1 by this matrix, for a total of 75,000 scalar multiplications.
- Thus, computing the product according to the first parenthesization is 10 times faster.

Formal Problem Definition

Specifying the Details

- We state the **matrix-chain multiplication problem** as follows: given a chain $\langle A_1, A_2, \dots, A_n \rangle$ of n matrices, where for $i = 1, 2, \dots, n$, matrix A_i has dimension $p_{i-1} \times p_i$, fully parenthesize the product $A_1 A_2 \dots A_n$ in a way that minimizes the number of scalar multiplications.

Formal Problem Definition

Specifying the Details

- We state the **matrix-chain multiplication problem** as follows: given a chain $\langle A_1, A_2, \dots, A_n \rangle$ of n matrices, where for $i = 1, 2, \dots, n$, matrix A_i has dimension $p_{i-1} \times p_i$, fully parenthesize the product $A_1 A_2 \dots A_n$ in a way that minimizes the number of scalar multiplications.
- The input is the sequence of dimensions $\langle p_0, p_1, p_2, \dots, p_n \rangle$.

Formal Problem Definition

Specifying the Details

- We state the **matrix-chain multiplication problem** as follows: given a chain $\langle A_1, A_2, \dots, A_n \rangle$ of n matrices, where for $i = 1, 2, \dots, n$, matrix A_i has dimension $p_{i-1} \times p_i$, fully parenthesize the product $A_1 A_2 \dots A_n$ in a way that minimizes the number of scalar multiplications.
- The input is the sequence of dimensions $\langle p_0, p_1, p_2, \dots, p_n \rangle$.
- The matrix-chain multiplication problem does not entail actually multiplying matrices.

Formal Problem Definition

Specifying the Details

- We state the **matrix-chain multiplication problem** as follows: given a chain $\langle A_1, A_2, \dots, A_n \rangle$ of n matrices, where for $i = 1, 2, \dots, n$, matrix A_i has dimension $p_{i-1} \times p_i$, fully parenthesize the product $A_1 A_2 \dots A_n$ in a way that minimizes the number of scalar multiplications.
- The input is the sequence of dimensions $\langle p_0, p_1, p_2, \dots, p_n \rangle$.
- The matrix-chain multiplication problem does not entail actually multiplying matrices.
- The goal is only to determine an order for multiplying matrices that has the lowest cost.

Formal Problem Definition

Specifying the Details

- We state the **matrix-chain multiplication problem** as follows: given a chain $\langle A_1, A_2, \dots, A_n \rangle$ of n matrices, where for $i = 1, 2, \dots, n$, matrix A_i has dimension $p_{i-1} \times p_i$, fully parenthesize the product $A_1 A_2 \dots A_n$ in a way that minimizes the number of scalar multiplications.
- The input is the sequence of dimensions $\langle p_0, p_1, p_2, \dots, p_n \rangle$.
- The matrix-chain multiplication problem does not entail actually multiplying matrices.
- The goal is only to determine an order for multiplying matrices that has the lowest cost.
- Typically, the time invested in determining this optimal order is more than paid for by the time saved later on when actually performing the matrix multiplications (such as performing only 7500 scalar multiplications instead of 75,000).

Counting The Number of Parenthesizations

There are Exponentially Many of Them

- Let us first convince ourselves that exhaustively checking all possible parenthesizations is not an efficient algorithm.

Counting The Number of Parenthesizations

There are Exponentially Many of Them

- Let us first convince ourselves that exhaustively checking all possible parenthesizations is not an efficient algorithm.
- Denote the number of alternative parenthesizations of a sequence of n matrices by $P(n)$.

Counting The Number of Parenthesizations

There are Exponentially Many of Them

- Let us first convince ourselves that exhaustively checking all possible parenthesizations is not an efficient algorithm.
- Denote the number of alternative parenthesizations of a sequence of n matrices by $P(n)$.
- When $n = 1$, the sequence consists of just one matrix, and therefore there is only one way to fully parenthesize the matrix product.

Counting The Number of Parenthesizations

There are Exponentially Many of Them

- Let us first convince ourselves that exhaustively checking all possible parenthesizations is not an efficient algorithm.
- Denote the number of alternative parenthesizations of a sequence of n matrices by $P(n)$.
- When $n = 1$, the sequence consists of just one matrix, and therefore there is only one way to fully parenthesize the matrix product.
- When $n \geq 2$, a fully parenthesized matrix product is the product of two fully parenthesized matrix sub-products, and the split between the two sub-products may occur between the k^{th} and $(k + 1)^{\text{st}}$ matrices for any $k = 1, 2, \dots, n - 1$.

Counting The Number of Parenthesizations

There are Exponentially Many of Them

- Let us first convince ourselves that exhaustively checking all possible parenthesizations is not an efficient algorithm.
- Denote the number of alternative parenthesizations of a sequence of n matrices by $P(n)$.
- When $n = 1$, the sequence consists of just one matrix, and therefore there is only one way to fully parenthesize the matrix product.
- When $n \geq 2$, a fully parenthesized matrix product is the product of two fully parenthesized matrix sub-products, and the split between the two sub-products may occur between the k^{th} and $(k + 1)^{\text{st}}$ matrices for any $k = 1, 2, \dots, n - 1$.
- Thus, we obtain the recurrence

$$P(n) = \begin{cases} 1 & \text{if } n = 1, \\ \sum_{k=1}^{n-1} P(k)P(n-k) & \text{if } n \geq 2. \end{cases}$$

Counting The Number of Parenthesizations

There are Exponentially Many of Them

- Let us first convince ourselves that exhaustively checking all possible parenthesizations is not an efficient algorithm.
- Denote the number of alternative parenthesizations of a sequence of n matrices by $P(n)$.
- When $n = 1$, the sequence consists of just one matrix, and therefore there is only one way to fully parenthesize the matrix product.
- When $n \geq 2$, a fully parenthesized matrix product is the product of two fully parenthesized matrix sub-products, and the split between the two sub-products may occur between the k^{th} and $(k + 1)^{\text{st}}$ matrices for any $k = 1, 2, \dots, n - 1$.
- Thus, we obtain the recurrence

$$P(n) = \begin{cases} 1 & \text{if } n = 1, \\ \sum_{k=1}^{n-1} P(k)P(n-k) & \text{if } n \geq 2. \end{cases}$$

- The solution to this recurrence is $\Omega(2^n)$, and hence, an exhaustive search makes for a poor strategy when determining how to optimally parenthesize a matrix chain.

Counting The Number of Parenthesizations

There are Exponentially Many of Them

- Let us first convince ourselves that exhaustively checking all possible parenthesizations is not an efficient algorithm.
- Denote the number of alternative parenthesizations of a sequence of n matrices by $P(n)$.
- When $n = 1$, the sequence consists of just one matrix, and therefore there is only one way to fully parenthesize the matrix product.
- When $n \geq 2$, a fully parenthesized matrix product is the product of two fully parenthesized matrix sub-products, and the split between the two sub-products may occur between the k^{th} and $(k + 1)^{\text{st}}$ matrices for any $k = 1, 2, \dots, n - 1$.
- Thus, we obtain the recurrence

$$P(n) = \begin{cases} 1 & \text{if } n = 1, \\ \sum_{k=1}^{n-1} P(k)P(n-k) & \text{if } n \geq 2. \end{cases}$$

- The solution to this recurrence is $\Omega(2^n)$, and hence, an exhaustive search makes for a poor strategy when determining how to optimally parenthesize a matrix chain.
- If you are wondering the solution to this recurrence of some value of n , you can convert the recurrence to an $O(n^2)$ dynamic programming algorithm.

Characterizing the Optimal Solution

Details

- Let $A_{i:j}$, where $i \leq j$, denote the matrix that results from evaluating the product $A_i A_{i+1} A_j$.

Characterizing the Optimal Solution

Details

- Let $A_{i:j}$, where $i \leq j$, denote the matrix that results from evaluating the product $A_i A_{i+1} \dots A_j$.
- If the problem is nontrivial, that is, $i < j$, then to parenthesize the product $A_i A_{i+1} \dots A_j$, the product must split between A_k and A_{k+1} for some integer k in the range $i \leq k < j$. That is, for some value of k , first compute the matrices $A_{i:k}$ and $A_{k+1:j}$, and then multiply them together to produce the final product $A_{i:j}$.

Characterizing the Optimal Solution

Details

- Let $A_{i:j}$, where $i \leq j$, denote the matrix that results from evaluating the product $A_i A_{i+1} A_j$.
- If the problem is nontrivial, that is, $i < j$, then to parenthesize the product $A_i A_{i+1} A_j$, the product must split between A_k and A_{k+1} for some integer k in the range $i \leq k < j$. That is, for some value of k , first compute the matrices $A_{i:k}$ and $A_{k+1:j}$, and then multiply them together to produce the final product $A_{i:j}$.
- The cost of parenthesizing this way is the cost of computing the matrix $A_{i:k}$, plus the cost of computing $A_{k+1:j}$, plus the cost of multiplying them together.

Characterizing the Optimal Solution

Details

- Let $A_{i:j}$, where $i \leq j$, denote the matrix that results from evaluating the product $A_i A_{i+1} \dots A_j$.
- If the problem is nontrivial, that is, $i < j$, then to parenthesize the product $A_i A_{i+1} \dots A_j$, the product must split between A_k and A_{k+1} for some integer k in the range $i \leq k < j$. That is, for some value of k , first compute the matrices $A_{i:k}$ and $A_{k+1:j}$, and then multiply them together to produce the final product $A_{i:j}$.
- The cost of parenthesizing this way is the cost of computing the matrix $A_{i:k}$, plus the cost of computing $A_{k+1:j}$, plus the cost of multiplying them together.
- **Optimal Substructure:** Suppose that to optimally parenthesize $A_i A_{i+1} \dots A_j$, you split the product between A_k and A_{k+1} .

Characterizing the Optimal Solution

Details

- Let $A_{i:j}$, where $i \leq j$, denote the matrix that results from evaluating the product $A_i A_{i+1} \dots A_j$.
- If the problem is nontrivial, that is, $i < j$, then to parenthesize the product $A_i A_{i+1} \dots A_j$, the product must split between A_k and A_{k+1} for some integer k in the range $i \leq k < j$. That is, for some value of k , first compute the matrices $A_{i:k}$ and $A_{k+1:j}$, and then multiply them together to produce the final product $A_{i:j}$.
- The cost of parenthesizing this way is the cost of computing the matrix $A_{i:k}$, plus the cost of computing $A_{k+1:j}$, plus the cost of multiplying them together.
- **Optimal Substructure:** Suppose that to optimally parenthesize $A_i A_{i+1} \dots A_j$, you split the product between A_k and A_{k+1} .
- Then the way you parenthesize both the "prefix" sub-chain $A_i \dots A_k$, and the "suffix" sub-chain $A_{k+1} \dots A_j$ within this optimal parenthesization of $A_i \dots A_j$ must be an optimal parenthesizations.

Characterizing the Optimal Solution

Details

- Let $A_{i:j}$, where $i \leq j$, denote the matrix that results from evaluating the product $A_i A_{i+1} \dots A_j$.
- If the problem is nontrivial, that is, $i < j$, then to parenthesize the product $A_i A_{i+1} \dots A_j$, the product must split between A_k and A_{k+1} for some integer k in the range $i \leq k < j$. That is, for some value of k , first compute the matrices $A_{i:k}$ and $A_{k+1:j}$, and then multiply them together to produce the final product $A_{i:j}$.
- The cost of parenthesizing this way is the cost of computing the matrix $A_{i:k}$, plus the cost of computing $A_{k+1:j}$, plus the cost of multiplying them together.
- **Optimal Substructure:** Suppose that to optimally parenthesize $A_i A_{i+1} \dots A_j$, you split the product between A_k and A_{k+1} .
- Then the way you parenthesize both the "prefix" sub-chain $A_i \dots A_k$, and the "suffix" sub-chain $A_{k+1} \dots A_j$ within this optimal parenthesization of $A_i \dots A_j$ must be an optimal parenthesizations.
- To build an optimal solution to an instance of the matrix-chain multiplication problem, we must split the problem into two sub-problems (optimally parenthesizing $A_i \dots A_k$ and $A_{k+1} \dots A_j$), find optimal solutions to the two sub-problem instances, and then combine these optimal sub-problem solutions.

Characterizing the Optimal Solution

Details

- Let $A_{i:j}$, where $i \leq j$, denote the matrix that results from evaluating the product $A_i A_{i+1} \dots A_j$.
- If the problem is nontrivial, that is, $i < j$, then to parenthesize the product $A_i A_{i+1} \dots A_j$, the product must split between A_k and A_{k+1} for some integer k in the range $i \leq k < j$. That is, for some value of k , first compute the matrices $A_{i:k}$ and $A_{k+1:j}$, and then multiply them together to produce the final product $A_{i:j}$.
- The cost of parenthesizing this way is the cost of computing the matrix $A_{i:k}$, plus the cost of computing $A_{k+1:j}$, plus the cost of multiplying them together.
- **Optimal Substructure:** Suppose that to optimally parenthesize $A_i A_{i+1} \dots A_j$, you split the product between A_k and A_{k+1} .
- Then the way you parenthesize both the "prefix" sub-chain $A_i \dots A_k$, and the "suffix" sub-chain $A_{k+1} \dots A_j$ within this optimal parenthesization of $A_i \dots A_j$ must be an optimal parenthesizations.
- To build an optimal solution to an instance of the matrix-chain multiplication problem, we must split the problem into two sub-problems (optimally parenthesizing $A_i \dots A_k$ and $A_{k+1} \dots A_j$), find optimal solutions to the two sub-problem instances, and then combine these optimal sub-problem solutions.
- To ensure that we have examined the optimal split, we must consider all possible splits.

A Recursive Solution

Deriving A Recurrence From The Characterization of Optimal Solutions

- Let $m[i, j]$ be the minimum number of scalar multiplications needed to compute $A_{i:j}$.

A Recursive Solution

Deriving A Recurrence From The Characterization of Optimal Solutions

- Let $m[i, j]$ be the minimum number of scalar multiplications needed to compute $A_{i:j}$.
- For the full problem, the lowest-cost way to compute $A_{1:n}$ is thus $m[1, n]$.

A Recursive Solution

Deriving A Recurrence From The Characterization of Optimal Solutions

- Let $m[i, j]$ be the minimum number of scalar multiplications needed to compute $A_{i:j}$.
- For the full problem, the lowest-cost way to compute $A_{1:n}$ is thus $m[1, n]$.
- We can define $m[i, j]$ recursively as follows. If $i = j$, no scalar multiplications are necessary to compute the product. Thus, $m[i, i] = 0$ for $i = 1, 2, \dots, n$.

A Recursive Solution

Deriving A Recurrence From The Characterization of Optimal Solutions

- Let $m[i, j]$ be the minimum number of scalar multiplications needed to compute $A_{i:j}$.
- For the full problem, the lowest-cost way to compute $A_{1:n}$ is thus $m[1, n]$.
- We can define $m[i, j]$ recursively as follows. If $i = j$, no scalar multiplications are necessary to compute the product. Thus, $m[i, i] = 0$ for $i = 1, 2, \dots, n$.
- To compute $m[i, j]$ when $i < j$, we take advantage of the structure of an optimal solution.

A Recursive Solution

Deriving A Recurrence From The Characterization of Optimal Solutions

- Let $m[i, j]$ be the minimum number of scalar multiplications needed to compute $A_{i:j}$.
- For the full problem, the lowest-cost way to compute $A_{1:n}$ is thus $m[1, n]$.
- We can define $m[i, j]$ recursively as follows. If $i = j$, no scalar multiplications are necessary to compute the product. Thus, $m[i, i] = 0$ for $i = 1, 2, \dots, n$.
- To compute $m[i, j]$ when $i < j$, we take advantage of the structure of an optimal solution.
- Suppose an optimal parenthesization splits the product $A_i A_{i+1} \dots A_j$ between A_k and A_{k+1} .

A Recursive Solution

Deriving A Recurrence From The Characterization of Optimal Solutions

- Let $m[i, j]$ be the minimum number of scalar multiplications needed to compute $A_{i:j}$.
- For the full problem, the lowest-cost way to compute $A_{1:n}$ is thus $m[1, n]$.
- We can define $m[i, j]$ recursively as follows. If $i = j$, no scalar multiplications are necessary to compute the product. Thus, $m[i, i] = 0$ for $i = 1, 2, \dots, n$.
- To compute $m[i, j]$ when $i < j$, we take advantage of the structure of an optimal solution.
- Suppose an optimal parenthesization splits the product $A_i A_{i+1} \dots A_j$ between A_k and A_{k+1} .
- Then, $m[i, j]$ equals the minimum cost $m[i, k]$ for computing the subproduct $A_{i:k}$, plus the minimum cost $m[k+1, j]$ for computing the subproduct $A_{k+1:j}$, plus the cost of multiplying these two matrices together, which is $p_{i-1} p_k p_j$.

A Recursive Solution

Deriving A Recurrence From The Characterization of Optimal Solutions

- Let $m[i, j]$ be the minimum number of scalar multiplications needed to compute $A_{i:j}$.
- For the full problem, the lowest-cost way to compute $A_{1:n}$ is thus $m[1, n]$.
- We can define $m[i, j]$ recursively as follows. If $i = j$, no scalar multiplications are necessary to compute the product. Thus, $m[i, i] = 0$ for $i = 1, 2, \dots, n$.
- To compute $m[i, j]$ when $i < j$, we take advantage of the structure of an optimal solution.
- Suppose an optimal parenthesization splits the product $A_i A_{i+1} \dots A_j$ between A_k and A_{k+1} .
- Then, $m[i, j]$ equals the minimum cost $m[i, k]$ for computing the subproduct $A_{i:k}$, plus the minimum cost $m[k+1, j]$ for computing the subproduct $A_{k+1:j}$, plus the cost of multiplying these two matrices together, which is $p_{i-1} p_k p_j$.
- Thus, $m[i, j] = m[i, k] + m[k+1, j] + p_{i-1} p_k p_j$. This equation assumes that you know the value of k . But you don't. You have to try all possible values of $k = i, i+1, \dots, j-1$.

A Recursive Solution

Deriving A Recurrence From The Characterization of Optimal Solutions

- Let $m[i, j]$ be the minimum number of scalar multiplications needed to compute $A_{i:j}$.
- For the full problem, the lowest-cost way to compute $A_{1:n}$ is thus $m[1, n]$.
- We can define $m[i, j]$ recursively as follows. If $i = j$, no scalar multiplications are necessary to compute the product. Thus, $m[i, i] = 0$ for $i = 1, 2, \dots, n$.
- To compute $m[i, j]$ when $i < j$, we take advantage of the structure of an optimal solution.
- Suppose an optimal parenthesization splits the product $A_i A_{i+1} \dots A_j$ between A_k and A_{k+1} .
- Then, $m[i, j]$ equals the minimum cost $m[i, k]$ for computing the subproduct $A_{i:k}$, plus the minimum cost $m[k+1, j]$ for computing the subproduct $A_{k+1:j}$, plus the cost of multiplying these two matrices together, which is $p_{i-1} p_k p_j$.
- Thus, $m[i, j] = m[i, k] + m[k+1, j] + p_{i-1} p_k p_j$. This equation assumes that you know the value of k . But you don't. You have to try all possible values of $k = i, i+1, \dots, j-1$.
- Thus, the recursive definition for the minimum cost of parenthesizing the product $A_i A_{i+1} \dots A_j$ becomes

$$m[i, j] = \begin{cases} 0 & \text{if } i = j, \\ \min\{m[i, k] + m[k+1, j] + p_{i-1} p_k p_j : i \leq k < j\} & \text{if } i < j. \end{cases}$$

Discussing Implementation and Computation of an Optimal Solution

Details

- The $m[i, j]$ values give the costs of optimal solutions to sub-problems.

Discussing Implementation and Computation of an Optimal Solution

Details

- The $m[i, j]$ values give the costs of optimal solutions to sub-problems.
- Let's define $s[i, j]$ to be a value of k at which you split the product $A_i A_{i+1} \dots A_j$ in an optimal parenthesization.

Discussing Implementation and Computation of an Optimal Solution

Details

- The $m[i, j]$ values give the costs of optimal solutions to sub-problems.
- Let's define $s[i, j]$ to be a value of k at which you split the product $A_i A_{i+1} \dots A_j$ in an optimal parenthesization.
- As we have seen before, a recursive algorithm based on the recurrence is no better than the brute-force method of checking each way of parenthesizing the product.

Discussing Implementation and Computation of an Optimal Solution

Details

- The $m[i, j]$ values give the costs of optimal solutions to sub-problems.
- Let's define $s[i, j]$ to be a value of k at which you split the product $A_i A_{i+1} \dots A_j$ in an optimal parenthesization.
- As we have seen before, a recursive algorithm based on the recurrence is no better than the brute-force method of checking each way of parenthesizing the product.
- Fortunately, there aren't all that many distinct sub-problems: just one sub-problem for each choice of i and j satisfying $1 \leq i \leq j \leq n$.

Discussing Implementation and Computation of an Optimal Solution

Details

- The $m[i, j]$ values give the costs of optimal solutions to sub-problems.
- Let's define $s[i, j]$ to be a value of k at which you split the product $A_i A_{i+1} \dots A_j$ in an optimal parenthesization.
- As we have seen before, a recursive algorithm based on the recurrence is no better than the brute-force method of checking each way of parenthesizing the product.
- Fortunately, there aren't all that many distinct sub-problems: just one sub-problem for each choice of i and j satisfying $1 \leq i \leq j \leq n$.
- Recall that the property of overlapping sub-problems is the second hallmark of when dynamic programming applies (the first hallmark being optimal substructure).

Pseudocode For Matrix-Chain Multiplication Algorithm

```
MATRIX-CHAIN-ORDER( $p, n$ )  
1  let  $m[1:n, 1:n]$  and  $s[1:n-1, 2:n]$  be new tables  
2  for  $i = 1$  to  $n$                                 // chain length 1  
3       $m[i, i] = 0$   
4  for  $l = 2$  to  $n$                                 //  $l$  is the chain length  
5      for  $i = 1$  to  $n - l + 1$                     // chain begins at  $A_i$   
6           $j = i + l - 1$                         // chain ends at  $A_j$   
7           $m[i, j] = \infty$   
8          for  $k = i$  to  $j - 1$                     // try  $A_{i:k} A_{k+1:j}$   
9               $q = m[i, k] + m[k + 1, j] + p_{i-1} p_k p_j$   
10             if  $q < m[i, j]$   
11                  $m[i, j] = q$                     // remember this cost  
12                  $s[i, j] = k$                     // remember this index  
13  return  $m$  and  $s$ 
```

Pseudocode For Printing Optimal Paranthesization

PRINT-OPTIMAL-PARENS(s, i, j)

```
1  if  $i == j$ 
2      print " $A$ " $i$ 
3  else print "("
4      PRINT-OPTIMAL-PARENS( $s, i, s[i, j]$ )
5      PRINT-OPTIMAL-PARENS( $s, s[i, j] + 1, j$ )
6      print ")"
```

An Earlier Example

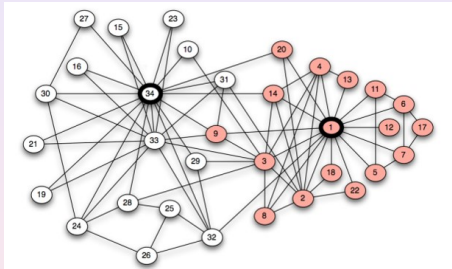


Figure: Real division of the Karate club. Reasonable graph partition algorithms put person 9 to the other group. Zachary traced the anomalous nature of node 9 to a fact that the network structure could not capture: at the time of the actual split, the person corresponding to node 9 was three weeks away from completing a four-year quest to obtain a black belt, which he could only do with the instructor (node 1).

Betweenness

Details of Girvan-Newman Method

We define our notion of traffic as follows.

- For each pair of nodes A and B in the graph that are connected by a path, we imagine having one unit of fluid flow along the edges from A to B .

Betweenness

Details of Girvan-Newman Method

We define our notion of traffic as follows.

- For each pair of nodes A and B in the graph that are connected by a path, we imagine having one unit of fluid flow along the edges from A to B .
- The flow between A and B divides itself evenly along all the possible shortest paths from A to B : so if there are k shortest paths from A and B , then $1/k$ units of flow pass along each one.

Betweenness

Details of Girvan-Newman Method

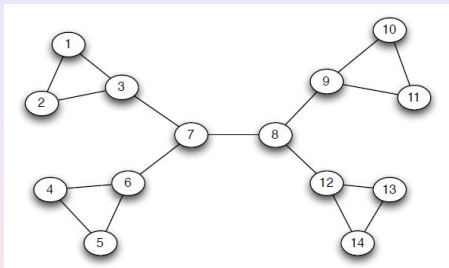
We define our notion of traffic as follows.

- For each pair of nodes A and B in the graph that are connected by a path, we imagine having one unit of fluid flow along the edges from A to B .
- The flow between A and B divides itself evenly along all the possible shortest paths from A to B : so if there are k shortest paths from A and B , then $1/k$ units of flow pass along each one.

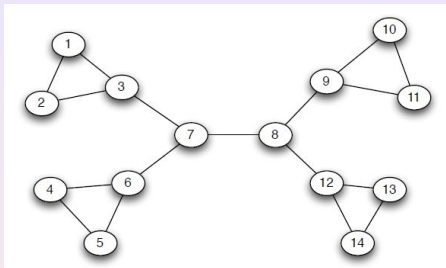
Definition

The **betweenness** of an edge is the total amount of flow it carries, counting flow between all pairs of nodes using this edge.

Explaining the Notion of Betweenness



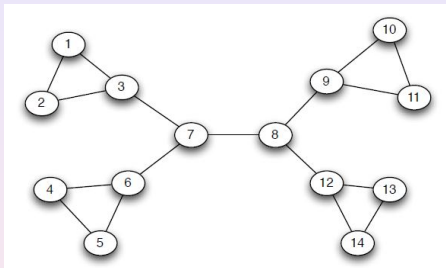
Explaining the Notion of Betweenness



Example

- Betweenness of 7 – 8: For each node A in the left half of the graph, and each node B in the right half of the graph, their full unit of flow passes through the 7 – 8 edge. No other flow uses this edge. So, betweenness is $7 \times 7 = 49$.

Explaining the Notion of Betweenness



Example

- Betweenness of 7 – 8: For each node A in the left half of the graph, and each node B in the right half of the graph, their full unit of flow passes through the 7 – 8 edge. No other flow uses this edge. So, betweenness is $7 \times 7 = 49$.
- Betweenness of 1 – 3: It carries all the flow from 1 to every other node except 2. No other flow uses this edge. So, its betweenness is 12. By symmetric reasoning, the other edges linked from 3, 6, 9, and 12 into their respective triangles have betweenness 12 as well.

The Girvan-Newman Method

Successively Deleting Edges of High Betweenness

- **(1)** Find the edge of highest betweenness (or multiple edges of highest betweenness, if there is a tie) and remove these edges from the graph. This may cause the graph to separate into multiple components. If so, this is the first level of regions in the partitioning of the graph.
- **(2)** Recalculate all betweennesses, and again remove the edge or edges of highest betweenness. This may break some of the existing components into smaller components; if so, these are regions nested within the larger regions.
- **(...)** Proceed in this way as long as edges remain in graph, in each step recalculating all betweennesses and removing the edge or edges of highest betweenness.

The Girvan-Newman Method

Successively Deleting Edges of High Betweenness

- **(1)** Find the edge of highest betweenness (or multiple edges of highest betweenness, if there is a tie) and remove these edges from the graph. This may cause the graph to separate into multiple components. If so, this is the first level of regions in the partitioning of the graph.
- **(2)** Recalculate all betweennesses, and again remove the edge or edges of highest betweenness. This may break some of the existing components into smaller components; if so, these are regions nested within the larger regions.
- **(...)** Proceed in this way as long as edges remain in graph, in each step recalculating all betweennesses and removing the edge or edges of highest betweenness.

Nested Structure

As the graph falls apart first into large pieces and then into smaller ones, the method naturally exposes a nested structure in the tightly-knit regions.

Example

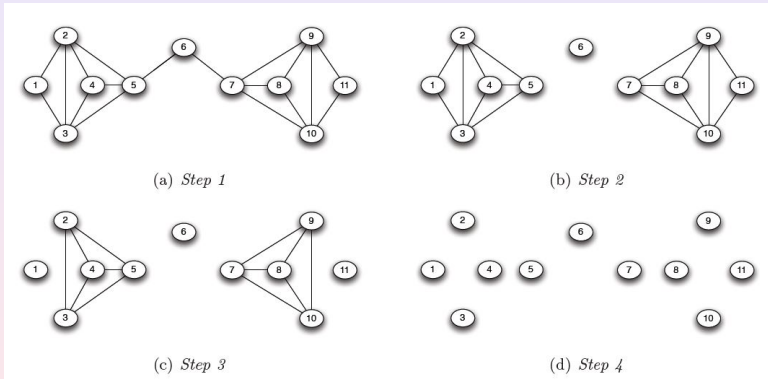


Figure: Steps of the Girvan-Newman method of graph partitioning

How to Compute Betweenness

Challenges

- In order to perform the Girvan-Newman method, we need a way to find the edges of highest betweenness in each step.

How to Compute Betweenness

Challenges

- In order to perform the Girvan-Newman method, we need a way to find the edges of highest betweenness in each step.
- This is done by computing all the betweennesses of all edges and then looking for the ones with the highest values.

How to Compute Betweenness

Challenges

- In order to perform the Girvan-Newman method, we need a way to find the edges of highest betweenness in each step.
- This is done by computing all the betweennesses of all edges and then looking for the ones with the highest values.
- The tricky part is that the definition of betweenness involves reasoning about the set of **all** the shortest paths between pairs of nodes.

How to Compute Betweenness

Challenges

- In order to perform the Girvan-Newman method, we need a way to find the edges of highest betweenness in each step.
- This is done by computing all the betweennesses of all edges and then looking for the ones with the highest values.
- The tricky part is that the definition of betweenness involves reasoning about the set of **all** the shortest paths between pairs of nodes.
- **Question:** Since there could be a very large number of such shortest paths, how can we efficiently compute betweenness without the overhead of actually listing out all such paths? This is crucial for implementing the method on a computer to work with datasets of any reasonable size.

How to Compute Betweenness

Challenges

- In order to perform the Girvan-Newman method, we need a way to find the edges of highest betweenness in each step.
- This is done by computing all the betweennesses of all edges and then looking for the ones with the highest values.
- The tricky part is that the definition of betweenness involves reasoning about the set of **all** the shortest paths between pairs of nodes.
- **Question:** Since there could be a very large number of such shortest paths, how can we efficiently compute betweenness without the overhead of actually listing out all such paths? This is crucial for implementing the method on a computer to work with datasets of any reasonable size.
- There is a clever way to compute betweennesses efficiently and it is based on the notion of Breadth-First-Search.

Clever way of Computing Betweenness

High-Level Description

- For each node, we will compute how the total flow from that node to all others is distributed over the edges. If we do this for every node, then we can simply add up the flows from all of them to get the betweennesses on every edge.

Clever way of Computing Betweenness

High-Level Description

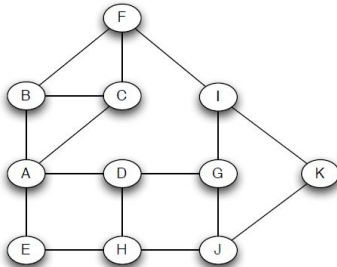
- For each node, we will compute how the total flow from that node to all others is distributed over the edges. If we do this for every node, then we can simply add up the flows from all of them to get the betweennesses on every edge.
- We will determine the flow from one node A to all other nodes in the graph with the three steps below.

Clever way of Computing Betweenness

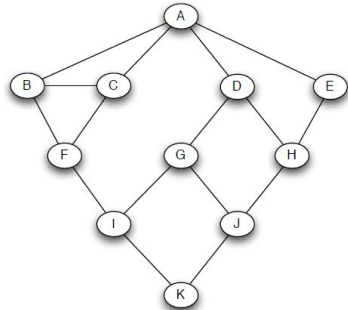
High-Level Description

- For each node, we will compute how the total flow from that node to all others is distributed over the edges. If we do this for every node, then we can simply add up the flows from all of them to get the betweennesses on every edge.
- We will determine the flow from one node A to all other nodes in the graph with the three steps below.
 - 1 Perform a breadth first search of the graph starting at A to compute the distances of each node from A .
 - 2 Determine the number of shortest paths from A to each other node.
 - 3 Based on these numbers, determine the amount of flow from A to all other nodes that uses each edge.

Explanation of the First Step of the Algorithm on an Example



(a) *A sample network*



(b) *Breadth-first search starting at node A*

Figure: The first step in the efficient method for computing betweenness values is to perform a BFS of the network. Here the results of BFS from node *A* are shown; over the course of the method, BFS is performed from each node in turn.

Computing the Number of Shortest Paths

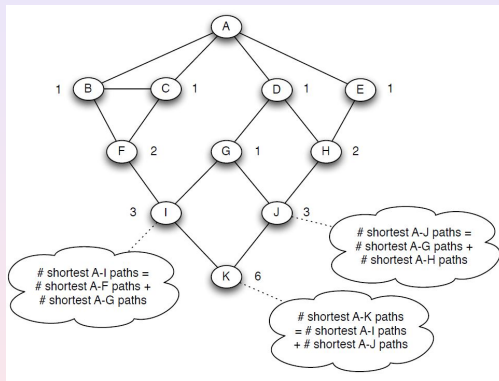


Figure: Each node in the first layer is a neighbor of A, and so it has only one shortest path from A. So we give each of these nodes a count of 1. The number of shortest paths to a node is the sum of the number of shortest paths to its neighbors in the layer above. So, we compute the number of shortest paths to each node by adding up counts of shortest paths, moving downward through the BFS structure.

Computing the Flow from A to Other Nodes

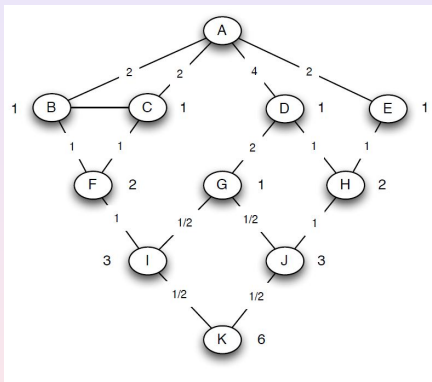


Figure: We compute the flow value of each edge in the bottom up fashion. For each node we add 1 to the incoming flow and distribute the flow to the edges going one layer up proportional with the count values of the neighbors computed in the previous step.